
Running oTree Experiments

Lab Runbook

Version 1.0 (Last updated: June 2026)

Vojtěch Zíka & Sára Laníková

<https://github.com/vojtechzika/labs-ujep>

Contents

Overview	2
A. One-Time Server-Machine Setup	3
A1 Verify / Install Python	3
A2 Verify / Install oTree	3
A3 Verify / Install Git	4
A4 Verify / Install Google Chrome	4
B. Preparing the Environment	5
B1 Download the Experiment from GitHub	5
B2 Start Virtual Environment	5
C. Running Locally	6
D. Running in the Lab	7
D1 Starting the oTree Server	7
D2 Admin Access to oTree	8
D3 Client Access to oTree	8
D4 Running a Session	10
D5 After the Session	12
E. Automation Using Batch Files	13
E1 Configuration	13
E2 First-Time Setup	14
E3 Starting a Session	14
E4 Ending a Session	14
E5 Backups	14

Overview

This document is the lab runbook for running oTree experiments at the Laboratories of Behavioral Studies at Jan Evangelista Purkyně University in Ústí nad Labem. It covers the full workflow from software installation to running a session with participants. The runbook is maintained at:

<https://github.com/vojtechzika/labs-ujep>

The repository also contains three supplementary folders referenced throughout this document:

- **_rooms** – room configuration folder to be placed in oTree project root, containing the participant label file (`lab_labels.txt`)
- **lab-bats** – automation scripts folder to be placed in oTree project root, automating the most error-prone parts of the session workflow
- **otree_kiosk** – kiosk launcher folder to be placed on each participant machine, containing the `.bat` files that open Chrome in kiosk mode for each seat

Software requirements: Windows, Python, oTree, Git, Chrome

Section	Description
A	Server setup – done once; involves installing all software needed to run oTree. In the lab, done by the lab manager. On your own machine, you must do it yourself.
B	Environment setup – done once before the study begins; downloading your oTree project and setting up a virtual environment.
C	Running locally – procedure for running oTree on your own computer for development and testing.
D	Running in the lab – procedure for running a lab session with multiple participants.
E	Automation – scripts that automate the most error-prone parts of the lab workflow, replacing most manual steps in B and D.

A. One-Time Server-Machine Setup

Open the Windows command line – search for `cmd.exe`.

A1. Verify / Install Python

Step 1) Verify what Python version (if any) your machine runs:

```
python --version
pip --version
```

Step 2) If Python is not present:

```
C:\Windows\System32>python --version
Python was not found; (...)
```

Download Python version compatible with your oTree software (the oTree-safe version is **Python 3.10**) from <https://www.python.org/downloads/>

During installation, check “**Add Python to PATH**” and accept all default options.

After installation, close and reopen CMD and repeat Step 1. You should see:

```
C:\Windows\System32>python --version
Python 3.10.11

C:\Windows\System32>pip --version
pip 23.0.1 from [path] (python 3.10)
```

IMPORTANT: If the version installed in the lab is not compatible with your software, please contact the lab manager. **DO NOT ATTEMPT TO REINSTALL THE PYTHON VERSION BY YOURSELF.**

A2. Verify / Install oTree

Step 1) Verify what oTree version (if any) your machine runs:

```
otree --version
```

Step 2) If oTree is not present:

```
pip install otree
```

This installs the latest version. If you need a specific version (e.g. to match an existing project):

```
pip install otree==5.10.3
```

Note: During installation you may see a message about a newer pip version being available. This is safe to ignore — updating pip is optional and not required for oTree to work.

After installation, repeat Step 1. You should see:

```
C:\Windows\System32>otree --version  
6.0.15
```

A3. Verify / Install Git

Step 1) Verify what Git version (if any) your machine runs:

```
git --version
```

Step 2) If Git is not present:

```
C:\Windows\System32>git --version  
'git' is not recognized as (...)
```

Download the most recent Git version from <https://git-scm.com>.

After installation (accept all default options), close and reopen CMD and repeat Step 1. You should see:

```
C:\Windows\System32>git --version  
git version 2.54.0.windows.1
```

A4. Verify / Install Google Chrome

This is a standard Google Chrome installation.

B. Preparing the Environment

(Once before the study begins)

B1. Download the Experiment from GitHub

Navigate to the folder where you want your oTree code to live, e.g. "C:\experiment":

```
cd "C:\experiment"
```

Download your experiment from GitHub. This is read-only – no login, SSH, or password is required for cloning. The only requirement is that the project at GitHub is **public**. You can try with the following project:

```
git clone https://github.com/vojtechzika/sequence-betting.git
```

Now the repository has been copied into "C:\experiment". Navigate to the subfolder containing oTree code (the folder containing `settings.py`; in this case `sequence-betting\otree`):

```
cd sequence-betting\otree
```

B2. Start Virtual Environment

From inside the project folder run:

```
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

IMPORTANT: After activation, (venv) must appear at the beginning of the command line prompt: (venv) C:\experiment\sequence-betting\otree>

In the folder with the oTree code you should now see a new folder named `venv`. This folder contains the virtual environment from which the server is run. It is **server-machine specific** – if you later run from a different machine, do not copy this folder; go through all steps from A to B2 again.

C. Running Locally

Follow this procedure to run a local test server for software development.

In a new CMD window, start the server:

```
cd "C:\experiment\sequence-betting\otree"  
.\venv\Scripts\activate  
otree devserver
```

IMPORTANT : Always activate venv before running oTree (`.\venv\Scripts\activate`). Running oTree without the venv may use a different globally installed version, causing database and software inconsistencies. To confirm the venv is active, the CMD prompt should begin with (venv), e.g.:

```
(venv) C:\experiment\sequence-betting\otree>
```

You should now see:

```
Open your browser to http://localhost:8000/  
To quit the server, press Control+C.
```

The CMD window is your server – as long as it is open, the oTree admin will be available on the printed address (usually `http://localhost:8000/`) that you access in Chrome as any other website.

IMPORTANT : Once you close the CMD window or press `Ctrl+C`, the server stops and you must start it again from Step 1, including the venv activation.

D. Running in the Lab

Running oTree in the lab adds complexity compared to local development: since clients run on different machines than the server, they cannot connect via `localhost` but must connect through the local network. The server machine and all client machines must be connected to the same network (which is usually the case in experimental labs).

Most of this setup is done by the lab manager, but you should understand the infrastructure and what your software must include to utilize it. Running oTree in the lab this way is the most effective approach.

D1. Starting the oTree Server

The procedure is the same as in Section C, except you now start a **production server** (`prodserver`) rather than a development server (`devserver`).

In the laboratory, there are two experimenter PCs with accounts `ControlRoom1` and `ControlRoom2`. Dedicate one of them as the server machine. (While it is possible to run the server from any computer on the network – including a laptop connected via WiFi – that meets the software requirements, we recommend using one of these dedicated machines.)

In a new CMD window:

```
cd "C:\experiment\sequence-betting\otree"  
.\venv\Scripts\activate  
set OTREE_PRODUCTION=1  
set OTREE_AUTH_LEVEL=STUDY  
otree prodserver
```

Setting `OTREE_PRODUCTION=1` disables the development warning banner / debug box that may otherwise appear at the participant pages.

Setting `OTREE_AUTH_LEVEL=STUDY` activates the login requirement for the admin panel, using the credentials defined in `settings.py` (`ADMIN_USERNAME` and `ADMIN_PASSWORD`). Without it, the admin panel is accessible without any login. **This is highly recommended in production, as anyone on the network could otherwise access the admin panel and interfere with the study.**

If you see the following message, the server is running:

```
Running prodserver  
timeoutworker is listening for messages through DB
```

IMPORTANT: Once the server is running, leave the machine untouched and do not use it for anything else. **Do not use it as a client machine** and do not use it to access the oTree admin panel. The less activity on the server machine, the lower the risk of it crashing or freezing during the experiment.

D2. Admin Access to oTree

Once the server is running, you can access the oTree admin panel from any machine on the same network. If you are in the laboratory using one of the experimenter PCs, the admin page will be at one of these addresses:

ControlRoom1: `http://192.168.32.22:8000`

ControlRoom2: `http://192.168.32.234:8000`

Log in with the credentials you set in `settings.py`.

If you are using a different PC or something does not work, find the server IP by running command `ipconfig` in CMD window on the server machine and look for the **IPv4 Address** (ignore all IPv6 addresses). Then open Chrome on any machine on the same network and navigate to:

```
http://[IPv4 ADDRESS]:8000/
```

From the admin panel you can *start and manage sessions and rooms*, *monitor session progress and payments*, and *download data* after each session.

WARNING: While you can open the oTree admin panel directly on the server machine, this is not recommended. You may accidentally shut down the server or cause it to crash or freeze. In the lab, there are two experimenter PCs. Use one to run the server and the other to manage the oTree admin panel. If your laptop is connected to the school network via WiFi, you can also access the admin panel from there.

D3. Client Access to oTree

Our lab identifies participants by seat rather than by oTree-generated links. Each participant machine connects to the oTree server via a predefined URL that includes a unique seat label (e.g. `Seat01`, `Seat02`, ...), so the server can identify which machine is connecting – and you know which label corresponds to which physical seat.

To prevent participants from navigating away from the experiment, the URL is opened automatically in Chrome **kiosk mode** – a fullscreen browser with no address bar or navigation controls. This is handled by a `.bat` file placed on each participant machine's Desktop, with separate launchers for each potential server (`ControlRoom1` and `ControlRoom2`).

Note: A `.bat` (batch) file is a plain text file containing a sequence of CMD commands that run automatically when double-clicked – Windows' simplest form of scripting. Instead of typing commands manually in CMD, you save them in a `.bat` file and execute them with a double-click. In this case, the `.bat` file launches Chrome with specific flags that enable kiosk mode and direct the browser to the correct oTree seat URL. To create a `.bat` file: open Notepad, paste the code, then go to *File* → *Save As*. Set *Save as type* to *All Files* and name the file with a `.bat` extension (e.g. `seat01.bat`). If you save it as a `.txt` file, it will not work.

The kiosk launchers should already be set up on all participant machines. If not, follow the setup procedure described at the end of this section or contact the lab manager.

For the kiosk launchers to successfully connect to your oTree server, your project must follow these conventions.

Labels

Participants are identified by seat labels defined in a plain text file `_rooms/lab_labels.txt`, where each line contains one label. The `_rooms` folder must be located in the root of your oTree project directory (i.e. the same folder as `settings.py`). The file is available in the lab GitHub repository (see Overview):

```
Seat01
Seat02
Seat03
...
```

Configuration of settings.py

Your `settings.py` must include the following room definition. It tells oTree to use the label file and sets the room name to `lab`, which is the name used in the kiosk URLs. You can copy and paste this directly into your project's `settings.py`:

```
ROOMS = [
    dict(
        name='lab',
        display_name='Laboratory',
        participant_label_file='_rooms/lab_labels.txt',
    ),
]
```

The `name` field must be set to `lab` – this value appears in the kiosk URL (e.g. `/room/lab/`) and must match exactly. The `display_name` can be changed freely as it is only used in the oTree admin panel. If your `settings.py` already contains a `ROOMS` variable, add the `dict(...)` entry to the existing list rather than creating a duplicate.

With this configuration, each kiosk opens Chrome with a URL of the following form:

```
http://SERVER_IP:8000/room/lab/?participant_label=Seat01
http://SERVER_IP:8000/room/lab/?participant_label=Seat02
...
```

The seat label is hard-coded in each terminal's `.bat` file, so no manual entry is required. Each machine automatically identifies itself to the oTree server using the label that matches its physical seat.

Creating Kiosk Launchers

Step 1) On each lab computer where participants will sit, create a folder called `otree_kiosk` in `C:\` and place a `.bat` file inside it. For example, for Seat 01, create `seat01.bat` with the following content. (A template `.bat` file is available in the `otree_kiosk` folder of the lab GitHub repository – see Overview.)

```
@echo off
start "" "C:\Program Files\Google\Chrome\Application\chrome.exe" ^
--kiosk ^
--no-first-run ^
--disable-session-crashed-bubble ^
--start-maximized ^
--user-data-dir="C:\otree_kiosk\Seat01" ^
"http://SERVER_IP:8000/room/lab/?participant_label=Seat01"
```

Change the `SERVER_IP` placeholder to the IP address of the server (see D2).

IMPORTANT: If using the multiline format with `^`, make sure there are **no spaces or characters after** `^` on any line – it must be the absolute last character. Invisible trailing spaces are a common cause of failure and are hard to spot in standard text editors.

Step 2) Create a desktop shortcut by doing one of the following: right-click `seat01.bat`, select *Show more options* → *Send to* → *Desktop (create shortcut)*; or right-click and drag `seat01.bat` to the Desktop and select *Create shortcut here*. Rename the shortcut to whatever you find descriptive.

Step 3) Repeat for all seats. On Seat 02, replace all occurrences of `Seat01` with `Seat02`, and so on.

You can create multiple `.bat` files in the `otree_kiosk` folder – one for each potential server IP address, in case you are not sure which experimenter PC will be used as the server.

Note: After launching the kiosk for the first time, Chrome will automatically create a `Seat01` folder inside `C:\otree_kiosk\`. This is expected – Chrome uses this folder to store an isolated browser profile for that seat, ensuring that sessions across different seats do not interfere with each other.

If the kiosk is launched before the server is running, Chrome will open in fullscreen and display a connection error with no warning beforehand. To close it, press `Alt+F4`.

D4. Running a Session

Step 1) Launch the server

Step 2) Launch the kiosks

Once the oTree server is running, double-click the kiosk launcher shortcut on the Desktop of each machine you wish to use, selecting the launcher dedicated to the server machine you are using. Chrome will open in kiosk mode and display a welcome screen (in the language configured in oTree):

```
Vitejte
Oznaceni ucastnika: Seat01
[Start]
```

If you see a connection error instead, the server is either not running or is running on a different IP address.

The participant label is filled in automatically from the URL address.

Note: The label displayed on the welcome screen can be changed to any other value listed in `lab_labels.txt` – for example, `Seat01` could be changed to `Seat02`, causing identity confusion in the data. Note that changing it to an arbitrary value (e.g. `SeatABC`) will cause oTree to show an error, but changing it to another valid label will not – so make sure participants do not fiddle with it.

Once all participants are seated, let them press **[Start]**. They will proceed to a waiting screen:

```
Prosim cekejte  
Čeka se na začatek vasi session.
```

They will remain there until you create the session in the admin panel.

Step 3) Open the Room

Log in to the admin panel and navigate to **Rooms**. Select *Laboratory* and choose your experiment in the **Session Config** field (this corresponds to the app name defined in `settings.py`).

WARNING : Before creating the session, always verify the participant count physically – walk through the lab or confirm that all participants are on the waiting screen. Do not rely solely on what oTree reports: the room page shows how many participants have pressed **[Start]**, but this number may be incomplete if some have not yet done so. The number you enter must match exactly the number of terminals where **[Start]** will be pressed:

- If **more** terminals press **[Start]** than the defined number, oTree will randomly select which terminals the experiment runs on. It may happen that the experiment runs on terminals with no participants while others are left waiting. This can be resolved by re-seating participants to the active terminals and closing the inactive kiosks, but this is disruptive and should be avoided.
- If **fewer** terminals press **[Start]** than the defined number, some participants will be unable to join. A workaround is to close the room and open a new one for those who could not join, but this creates a separate session with a separate dataset, which complicates data analysis and payment.

Once you are sure of the number, enter it in the form and press **Create**. The session goes live immediately: participants already on the waiting screen are automatically redirected to the first page of the experiment, and those who have not yet pressed **[Start]** will be taken there as soon as they do.

Note: Since this setup uses hard-coded seat links, you can ignore the *Start Links* section in the admin panel.

If you close the room, the session remains active but no new participants can join. Opening another room creates a new, independent session — participants who join through the new room will run separately from those already in the first. This can be useful in two situations: to run a second study simultaneously, or as a last resort when more participants showed up than the session was created for. Keep in mind that this results in two separate datasets and sessions, which may complicate data analysis and payment. Use this only when necessary.

Click **Monitor the session** in the green box to track the progress, data, and payments of the session.

ABSOLUTE DON'TS DURING THE SESSION

GOLDEN RULE: Do not touch the server machine. Leave it completely untouched during data collection. In particular, do not:

- Close the CMD window running the server (by clicking, pressing **Ctrl+C**, etc.)
- Edit oTree code or open/edit any files in the oTree source folder
- Open or edit the database file (`db.sqlite3`)

Touching `db.sqlite3` during an active session will almost certainly result in permanent, unrecoverable loss of all data.

Troubleshooting

- If a single participant has a problem (e.g. their kiosk freezes or their computer crashes), you can safely close and reopen just that one kiosk via its `.bat` file, or restart the computer and reopen the kiosk — this does not affect other participants, unless your study involves real-time participant interactions. The participant should be able to pick up where they left off.
- If the server is accidentally shut down, restart the computer and start the server again. The kiosks (which by now show a connection error) should refresh automatically once the server is back online; if not, refresh manually with **Ctrl+F5** or close and reopen them via the `.bat` file. Participants should be able to continue from where they left off, though this is not guaranteed to work correctly in all cases.

D5. After the Session

Once all participants have left the lab, follow these steps.

Step 1) Close the Room. In the admin panel, press **Close this room**.

Step 2) Export Data. Go to *Admin* → *Data* and export *All apps (wide format)* – this gives you the full dataset with all variables. For extra safety, you can also download the per-app data files individually.

Step 3) Close the Kiosks. Close all kiosk windows on participant machines using **Alt+F4**.

Step 4) Shut Down the Server. In the CMD window running the server, press **Ctrl+C** to stop it and close the CMD window.

Step 5) Back Up the Database File. Only once the server is shut down, create a backup of the database file. You can do this manually – locate `db.sqlite3` in your oTree project folder (the same folder as `settings.py`), then copy and paste it in your backup folder. However, it is safer to use the command line, as manual copy-paste can occasionally result in a corrupted file or an accidental deletion. Open a new CMD window, navigate to your oTree project folder, and run:

```
copy db.sqlite3 db.sqlite3_backup1
```

This creates a copy of `db.sqlite3` named `db.sqlite3_backup1` in the same folder. If you run multiple sessions, increment the number each time to avoid overwriting a previous backup (e.g. `db.sqlite3_backup2`, `db.sqlite3_backup3`, ...).

IMPORTANT: Never delete `db.sqlite3` after the session. Only delete it once you are certain all data has been exported and verified, or when setting up a fresh study.

E. Automation Using Batch Files

Running a session manually requires remembering a sequence of steps under time pressure. The lab scripts automate the entire process — setting up the environment, starting the server, and shutting it down cleanly after the session, including automatic database backup. If used properly, you will not need to type a single command during the experiment.

The `lab-bats` folder with the scripts is available at the lab GitHub repository (see Overview). It contains the following files:

- `setup-lab.bat` – run once before the study
- `RUN-SESSION.bat` – run before each session
- `END-SESSION.bat` – run after each session
- `lab-config.bat` – you may want to edit this file
- `_start-otree.bat` – do not run directly
- `_backup-db.bat` – do not run directly

E1. Configuration

The `lab-bats` folder must be located **directly inside your oTree project folder** (the one containing `settings.py`). This means you first need to copy your oTree project to the server machine, or clone it from GitHub. The expected folder structure is:

```
repo/
+-- otree/
    +-- settings.py
    +-- requirements.txt
    +-- db.sqlite3
    +-- venv/ (will be created by the .bat files)
    +-- _rooms/
        +-- lab_labels.txt
    +-- lab-bats/
        +-- setup-lab.bat
        +-- RUN-SESSION.bat
        +-- END-SESSION.bat
        +-- lab-config.bat
        +-- _start-otree.bat
        +-- _backup-db.bat
```

All settings are centralised in `lab-config.bat`. This is the **only file you may need to edit** (in Notepad), and only if your setup differs from the lab defaults:

Variable	Description	Default
PYTHON_VERSION	Python version used to create venv	3.10
OTREE_PORT	Server port	8000
OTREE_AUTH_LEVEL	Authentication level	STUDY
ROOM_NAME	Room name as defined in <code>settings.py</code>	lab
ROOM_LABEL_FILE	Path to participant label file	_rooms\lab_labels.txt
BACKUP_ROOT	Backup location	D:\otree-backups

IMPORTANT : The backup location is set to `D:\otree-backups`. If the server machine does not have a D: drive, the easiest fix is to change `BACKUP_ROOT` to `C:\otree-backups`. `ADMIN_USERNAME` and `ADMIN_PASSWORD` are expected to be defined in `settings.py`.

E2. First-Time Setup

Run `setup-lab.bat` once on the server machine before the first session. It will verify Python and oTree is installed, check that `settings.py`, `requirements.txt`, and the room label file exist, create the virtual environment in `otree\venv`, and install all dependencies from `requirements.txt`.

Run this only once before the study.

E3. Starting a Session

Double-click `RUN-SESSION.bat` before each session. It will verify the virtual environment and required files, check that no oTree server is already running, create a pre-session database backup, start the oTree production server, and open the oTree admin panel in the browser as a convenience check. As noted earlier, you should access the admin panel from a different machine during the session.

Run this before each session.

E4. Ending a Session

After closing the room and downloading the data (D5, steps 1 and 2), double-click `END-SESSION.bat`. It will stop the oTree server, close the CMD window, create a post-session database backup, and open the backup folder automatically as a convenience check.

E5. Backups

Two backups are created automatically – one before the session starts and one after it ends. Each is stored in:

`D:\otree-backups\<project-name>\<timestamp>\`

Each backup folder contains `db.sqlite3` (a full copy of the database) and `session-log.txt` (backup metadata: project name, timestamp, and source and backup paths).